



Accelerating Louvain community detection algorithm on graphic processing unit

Maryam Mohammadi¹ · Mahmood Fazlali² · Mehdi Hosseinzadeh^{3,4}

Accepted: 4 November 2020 / Published online: 18 November 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

The Louvain community detection algorithm is a hierarchal clustering method categorized in the NP-hard problem. Its execution time to find communities in large graphs is, therefore, a challenge. Parallelization is an effective solution for amortizing Louvain's execution time. In this paper, we propose an adaptive CUDA Louvain method (ACLM) algorithm that benefits from the graphic processing unit (GPU). ACLM uses the shared memory in GPU, as well as the optimal number of threads in the GPU blocks. These features minimize parallelization overhead and accelerate the calculation of modularity parameters. The proposed algorithm allocates threads to each block based on the number of required streaming multiprocessors (SMs) and warps on GPU. The implementation results show that ACLM can effectively accelerate the execution time by 77% compared to the competitive method in the large graph benchmarks.

Keywords Louvain community detection · Parallel processing · Modularity · CUDA

✉ Mahmood Fazlali
Fazlali@sbu.ac.ir

Maryam Mohammadi
mohammadi.maryam@srbiau.ac.ir

Mehdi Hosseinzadeh
hosseinzadeh.m@iums.ac.ir

¹ Department of Computer Engineering, Science and Research Branch, Islamic Azad University, Tehran, Iran

² Department of Computer and Data Sciences, Faculty of Mathematical Sciences, Shahid Beheshti University, Tehran, Iran

³ Institute of Research and Development, Duy Tan University, Da Nang 550000, Vietnam

⁴ Mental Health Research Center, Psychosocial Health Research Institute, Iran University of Medical Sciences, Tehran, Iran

1 Introduction

Community detection is an important task in analyzing and decomposing of a network (graph). The community detection in a network is characterized as the finding of clusters whose intra-edges are much denser than inter-edges [1]. Community detection in large networks is a time-consuming and computationally complex task due to the size variations and unavailability of the number of communities [2].

Modularity [3, 4] is a quality metric for the assessment of community detection algorithms. Modularity is the ratio of intra-edges of a community to the sum of inter-edges in a randomly distributed edge graph. A higher modularity value indicates a better quality for community detection. Blondel et al. [6] proposed to use a hierarchical clustering method in which the modularity (Q) is computed in each step of the algorithm. This method, which is known as Louvain in many commercial cyberspace applications, requires to perform an optimization on an NP-Hard problem to find the modularity [5]. To solve the computational complexity, several fast heuristic methods have been proposed for community detection in large-scale graphs.

Parallelization is a suitable technique to reduce the algorithm's execution time [7–13]. The main objective, however, has been parallelization of calculating the modularity in the Louvain algorithm. This is done by employing the maximal parallelization power of the platform where a node (or community) is merged with its neighbour node [7, 8].

Fazlali et al. [11] introduced a new parallel approach for community detection based on the Louvain method on multicore systems. This method can be improved by using GPU cores to calculate the modularity of merging communities. Therefore, in this study, the authors introduce a new community detection algorithm named adaptive CUDA Louvain method (ACLM). The method uses both CPU and GPU together in a computational heterogeneous processing model. The proposed ACLM algorithm calculates the modularity of merging neighbour communities in the graph by available cores on the GPU blocks. This method reduces the time of community detection using GPU cores, and we achieved better results compared to competing methods. The proposed ACLM algorithm allocates the optimal number of threads to each block in the GPU and uses the shared memory in GPU blocks to calculate modularity resulting from the merging of two neighbour communities. It calculates the number of required streaming processors (SMs) to allocate threads to each block based on the multiplicand of warps. In this way, the algorithm can reduce the execution time from 45 to 77% in large benchmarks compared to previous parallel algorithms.

The rest of the paper is organized as follows. In Sect. 2, backgrounds on community detection, modularity criteria for evaluating the quality of a community and Louvain method are described. In Sect. 3, previous studies targetted parallelization of Louvain community detection algorithm are reviewed. The proposed ACLM algorithm is introduced in Sect. 4. Results, performance evaluation and comparisons to competing approaches are presented in Sect. 5. Finally, Sect. 6 concludes the paper.

2 Background

A graph of real systems does not have a regular structure, and it mostly has disordered elements. The pattern of an irregular graph is a random graph that was introduced by Erdos and Reyni [14], in which the probability of the existence of an edge between two nodes is the same for all possible combinations of each two nodes. The distribution of edges between two nodes in a random graph is highly homogeneous. For example, the distribution of a node neighbour (Degree) is the same and is binomial. Therefore, most of the nodes have the same or equal distribution. Real networks are not random graphs. They have heterogeneity in their structure, and the variety and extensiveness of their degree are exceedingly high. This means there exists an increased number of nodes with a low degree alongside several nodes with high degrees. Real networks normally consist of communities that are composed of smaller communities. Community detection in a multipart network have interesting applications. For instance, in large shopping networks whose customers are connected to the products they purchase, there exists the ability to classify customers based on the type of products they mostly purchase. This feature can be used in targeted advertisement and recommender systems to recommend items to customers for their future purchases [15].

There are several definitions for a community in a graph. Therefore, it is not surprising that there exist several solutions in different articles for community detection. Thus, multiple criteria have been proposed for community detection. In the most of the cases, these criteria are indirect definitions that use some of the features of a graph in an algorithm, which results in the detection of the community. One of the most commonly used criteria is modularity, which we will discuss in the following section.

Based on one of the basic definitions that researchers have agreed upon, a graph has a community structure if it is different from an irregular graph. Therefore, we can define a null or random graph. We use this graph as a comparing condition to decide whether the graph has a structural community or not. The most popular random model is introduced by Newman and Girvan [16]. It consists of a randomized version of an original graph whose edges are rewired at random. This is under the constraint that the expected degree of each node matches the degree of the node in the original graph. This null model is the main concept behind the modularity, which is the global criteria for the definition of a community and is used as a quality function in the Louvain method. The quality function or “modularity” Q is defined as follows. Given Eq. (1), the modularity depends on the total weight of all edges within a particular community and the total weight of all edges incident on each community C .

$$Q = \sum_{C=1}^{n_c} \left[\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right] \quad (1)$$

In Eq. (1), n_c is the number of communities, l_c is the number of all edges which connect vertices of community C , d_c is the sum of the degree of nodes in

community C , and m is total weights of all edges in the network. In this formula, the first term indicates total edges inside the community, while the second term indicates the edges that are expected to exist for each node in a random graph. That degree of each node matches the degree of the node in the original graph.

Hence, a subgraph is a community, if the modularity value is positive. The maximum modularity value of a graph can be higher if the graph size or the number of its independent clusters are increased. Therefore, modularity is not suitable for comparing the quality structure of communities in the graph that have different dimensions and should not be employed in such cases. The modularity of the whole graph that is considered to be a community unit is zero. Holmström et al. [17] showed that the distribution of modularity value between communities of different graphs consisting of both real and simulated graphs (consists of random graphs whose community structure is not clear) has stable characteristics. It conforms to the existing communities in clusters of approximately the same size.

In community detection algorithms, links typically have no direction, and it is assumed that graphs are undirected. Disregarding the direction of edges helps significantly in improving the partitioning of the system, but it is notable that in some cases disregarding the direction of edges could lead to confusing and unclear results [14].

One of the most important community detection algorithms based on the modularity is Louvain algorithm. This algorithm consists of two phases that are repeated iteratively. First, each node in the network is considered a community. In other words, each node is assigned a distinct community. Therefore, in each initial partition, the number of communities is the same as the number of nodes. Afterwards, for each node i , all of its j neighbours are checked and modularity is calculated for each situation where node i can be removed from its own community and moved to the communities of each one of its neighbour nodes. Then, node i is added to the community that has the most benefit. If the benefit is not positive, node i remains in its own community. This process is repeated continuously for each node, up to the point that no further improvements can be made. We should emphasize the fact that a node can be checked multiple times. The first phase is finished once the local maximum modularity is obtained. This is finished once no further moves can be made, which improves the modularity. We should also consider that the result of the algorithm depends on the order of the nodes that are being checked. The modularity benefit ΔQ from moving node i to community C is calculated using Eq. (2):

$$\Delta Q = \left[\frac{\sum_{in} + K_{i,in}}{2_m} - \left(\frac{\sum_{tot} + K_i}{2_m} \right)^2 \right] - \left[\frac{\sum_{in}}{2_m} - \left(\frac{\sum_{tot}}{2_m} \right)^2 - \left(\frac{K_i}{2_m} \right)^2 \right] \quad (2)$$

In Eq. (2), $\sum_{in}$ is the sum of the weights of the inner edges of community C , and $\sum_{tot}$ is the sum of all edges which have a vertex in community C . K_i is the sum of the weights of the edges which enter node i . $K_{i,in}$ is the sum of the weights of the edges that exist from node i to nodes inside the community C , and m is the total weight of all the edges in the network. The second phase of the algorithm

consists of building a new network whose nodes are communities that were found in the first phase. To do this task, the weight of edges between new nodes is obtained from the weight of edges between two communities. Edges between nodes in the same community lead to creating a loop in the community of a new network. After completing the second phase, the first phase can be applied to the weighted network again, and this process is repeated. Here, a run of the first and second phases is called a cycle in the Louvain method. Starting a new cycle, the reduced graph is fed to the first phase, so that new communities that maximize modularity are detected. Cycles continue as long as modularity is gained.

Considering the fact that in Louvain algorithm at first each node in the network is considered a community, the maximum number of supernodes would be equal to the size of the networks occurs at the first pass of the algorithm. Therefore, the most time-consuming part of the algorithm is the first pass. In the next passes, network size is decreased, so the execution time is decreased.

3 Related works

Community detection is a challenging problem in the analysis of real graphs in complex networks. Zhao [18] review several community detection methods and their theoretical properties, including graph cut methods, profile likelihoods, pseudo-likelihood methods, variational methods, belief propagation, spectral clustering, and semidefinite relaxations of the stochastic block model. Zhao also briefly discusses other research topics in community detection such as robust community detection, community detection with nodal covariates, and model selection. Also, CiteSpace is a freely available Java application for visualizing and analyzing trends and patterns in scientific literature. Khan and Niazi [19] have used CiteSpace for the visual analysis of pertinent peer-reviewed papers retrieved from Web of Science, devoted to network community detection over the last 25 years.

Chakraborty et al. [20] have presented a survey of the start-of-the-art metrics used for detecting and evaluating the community structure. Based on the type of communities detected, they have classified the quality metrics into different categories (non-overlapping and overlapping) and have performed experiments on synthetic and real networks to evaluate the usefulness of these metrics of the underlying community structure. In reviewing the quality metrics, authors have observed that the most popular and widely accepted metric in the literature of community analysis is Newman-Grivan's modularity, which also lays the foundation for other metrics. The community detection problems can then be formalized as a modularity maximization problem from the partitions of a graph. Although there are currently several methods to detect good approximations of the maximum modularity, among these methods, the Louvain method [6] is a well-known method that employs an iterative heuristic based on modularity maximization.

Louvain community detection method needs a powerful machine for the decomposition and analysis of large networks. This is usually expensive, and it is not affordable. Therefore, researchers have found alternatives for accelerating the process using parallel platforms. In each iteration of the Louvain algorithm,

if updating the community for nodes is done in parallel, it is not guaranteed that the resulting modularity is always positive. If the modularity between consecutive iterations is negative, there is no guarantee that the algorithm is concluded. Even if there exists a chance that the algorithm may not be concluded, there is the possibility that it may decrease the speed or create more iterations. Therefore, the number of algorithm iterations needed to reach the final results can be a good indicator for the effectiveness of parallelization in comparison to the sequential algorithms. Also, by increasing the modularity, if two nodes decide to be moved to each other's community, whose update may potentially conclude in both nodes, they can swap their communities without increasing the modularity. This scenario can also happen in a more general situation when a subset of nodes between two different communities (supernodes) change their communities without having any knowledge of each other's decision for movement. Another scenario that may also happen in a sequential algorithm is: if a community unit is divided into subsets with the same weight, its nodes may not have any motivation for merging with any of its other subsets. If all nodes from each of the subsets act upon merging a community unit, the modularity is positive [10].

Ghosh et al. [21] have presented a distributed memory implementation of Louvain algorithm for parallel graph community detection. Their approach begins with an arbitrarily partitioned distributed graph input and employs several heuristics to accelerate the computation of the different steps of the Louvain algorithm. Ozaki et al. [22] have proposed Louvain Prune algorithm that contributes to reducing Louvain runtime. Authors randomly choose the community of the neighbour node instead of considering all neighbouring communities during the Louvain method phase 1. Their experimental results show that Louvain Prune algorithm can be accelerated without quality deterioration on almost every kind of network generated by the LFR benchmark [23] and on all real networks that they tested.

Gutierrez et al. [24] have presented a modification of Louvain algorithm for a crisp network, which allows dealing with community detection problems in networks in which there exists additional information independent of its structure. This additional information is defined by a fuzzy measure, μ , which is aggregated to the weight of the classical relations among nodes when optimizing the modularity of clusters throughout the algorithm. Traag [25] has proposed a scheme where the nodes from a given community are moved to a random neighbour community, instead of the best neighbour community. This minor adjustment to the original Louvain method process leads to reductions in running time, especially when concentrating on the coarser partitions found by modularity, without losing much quality.

Recently, Carnival et al. [26] in order to overcome the cost of the first round of the Louvain method have introduced the Coarse-grained Vertex Clustering method (CoVeC) for community detection in sparse complex networks. The subsequent maximization of quality, as a modularity metric, is left to the Louvain method. CoVeC method preprocesses the original graph in order to forward a reduced size graph. According to their evaluation, CoVeC method outperforms the Louvain method and its variations, attaining a mean execution time reduction of 47% and a mean modularity reduction of only 0.4%.

In fact, until the present day, the Louvain method continues to be one of the most widely used tools for identifying communities in large-scale graphs quickly and efficiently. Considering the time complexity of the Louvain algorithm in large graphs, researchers have created methods to accelerate this algorithm using parallel platforms.

Bhowmick and Srinivasan [27] have presented a parallel algorithm for shared-memory platforms based on the maximization of modularity. This algorithm requires frequent synchronization of the communities across levels that is costly for parallelization. Results have shown that while their implementation is scalable and produces modularity values commensurate with those expected from a sequential case, the performance of this algorithm and variability of the results depend on the properties of the network as well as its size.

Several articles [7, 11, 28] have used OpenMP and shared memory structure in order to parallelize the Louvain method. Staudt and Meyerhenke [8] have proposed a method called PLM that uses label propagation for the Louvain method. The transition of nodes is done in parallel instead of sequentially in their method. Staudt and Meyerhenke [28] have proposed a flexible and distributed framework that benefits from overlapping the shared memory for community detection. In this framework, they use Parallel Label Propagation (PLP) that is introduced by Raghavan et al. [29] as the base algorithm and have proposed the parallel Louvain method with refinement (PLMR) as the final algorithm. PLMR adds refinement to each level, which results in an increase in modularity and a slight increase in the runtime of the algorithm.

Moradi et al. [7] have used a coarse-grained algorithm in order to decompose the Louvain algorithm to calculate modularity. This method uses a thread to calculate modularity to add the neighbours of the community to it, and it was faster in comparison with previous algorithms. Then, we proposed a new algorithm, named Adaptive Parallel Louvain Method (APLM) [11], for solving Louvain community detection in multicore systems that run in parallel across thread level. This algorithm allocates threads in parallel and dynamically for calculating modularity of adding eligible neighbour nodes to the community. This algorithm also tries to tackle the issue of dividing (compromising) between coarse-grained and fine-grained threads. To this end, the algorithm finds the number of idle cores and calculates the suitable number of threads to allocate cores for calculating modularity sigma. This resulted in improving algorithm speed-up while the criterion of the quality as modularity is approximately the same.

GPU by having extensive computational parallel abilities is a suitable choice for accelerating Louvain community detection algorithm in large networks. Cheong et al. [9] have introduced a parallel Louvain algorithm based on GPU implementation. This algorithm aims at three levels of parallelization in the Louvain method, and it can be executed using single-GPU and multiple-GPU architectures. The results of evaluating this algorithm on multiple large social networks show that it can accelerate community detection, but it results in a 3% reduction in the quality of community detection. Forster [13] shows that Louvain can be accelerated on GPU using CUDA library where the input graph matrix can be loaded in GPU memory, and it has dense edges, compared to the parallel Louvain implementation running

on eight threads on the same machine that has the same GPU. Their CUDA implementation provides absolute speed-up up to 12 times using the GeForce GTX 980M mobile GPU.

Shao et al. [30] have proposed the attractor method that uses the distance criteria instead of similarity in community detection. Reviews show that this method is slower than Parallel Louvain methods. Naim et al. [31] have proposed a GPU-based Louvain community detection method. This algorithm tries to hold an even load balance by scaling the number of threads assigned to each vertex, depending on its degree. This is achieved by partitioning the vertices into subsets depending on their degrees in the modularity optimization phase. These sets are processed in turn by computing and updating the destination community of each vertex in parallel. For each set, they use a different number of threads per vertex. Memory usage is the Weakness of this algorithm, and still, there is a need to present an efficient parallel Louvain algorithm for GPU-based platforms.

In this research, we have proposed the ACLM algorithm, which uses GPU to calculate the modularity sigma in parallel. The results of running the algorithm on multiple web-based networks show that it has better performance in comparison with the previous algorithms, while the quality of the resulted communities is comparable to them.

The main contribution of this paper is allocating threads to the block adaptively to calculate modularity. Another one is the dynamic usage of shared memory in each block. Implementation results demonstrate that defining the shared memory of each block and adaptively allocating threads to the blocks causes a reduction in the algorithm's execution time. This results in an effective parallelization of the Louvain method in comparison with the previous works.

4 The proposed adaptive CUDA Louvain method (ACLM) algorithm

Here we present our adaptive CUDA Louvain method (ACLM). The main idea of the algorithm is to accelerate the Louvain method using the computation power of GPU and management of the required memory for its processing threads. Nodes in large dense graphs have a high number of neighbours. Therefore, it can calculate the resulted modularity of merging a community with the neighbour networks in a fine-grained way using GPU.

In the ACLM algorithm, the calculation of modularity for merging a community with its neighbour community is done in parallel. This algorithm uses the base modularity in Eq. (3) [32]:

$$Q = \sum_i (e_{ii} - a_i^2) \quad (3)$$

In Eq. (3), i is the index of a community. e_{ii} is the ratio of the number of inner edges in the community i to all edges in the graph with a random distribution graph. Let e_{ij} be one-half of the fraction of edges in the network that connect vertices of the group i to those belonging to group j , so that $a_i = \sum_j e_{ij}$ is the ratio of the number of

edges that have at least one node in the community i to the number of all edges of the graph with random distribution.

ACLM algorithm calculates the modularity of the new graph (resulted from joining community i with neighbour community j) using GPU cores. This is done based on Eq. (3). Figure 1 presents the flowchart of calculating the modularity using GPU cores in parallel.

After merging two nodes, a supernode is formed which has inner edges (self-loops) inside itself. Hence, the new graph composes of supernodes. The sum of weights of the inner edges (self-loops) of each supernode is in vector in , and the sum of weights for all edges (weighted_degree) of each supernode is in vector tot in Algorithm 1. The vector in consists of e_{ii} and vector tot consists of a_i^2 . In other words, vector in equals $\sum_i e_{ii}$ and vector tot equals $\sum_i a_i^2$.

The blocking of threads and allocating the optimal number of threads in each block results in an effective reduction in the execution time of the ACLM algorithm

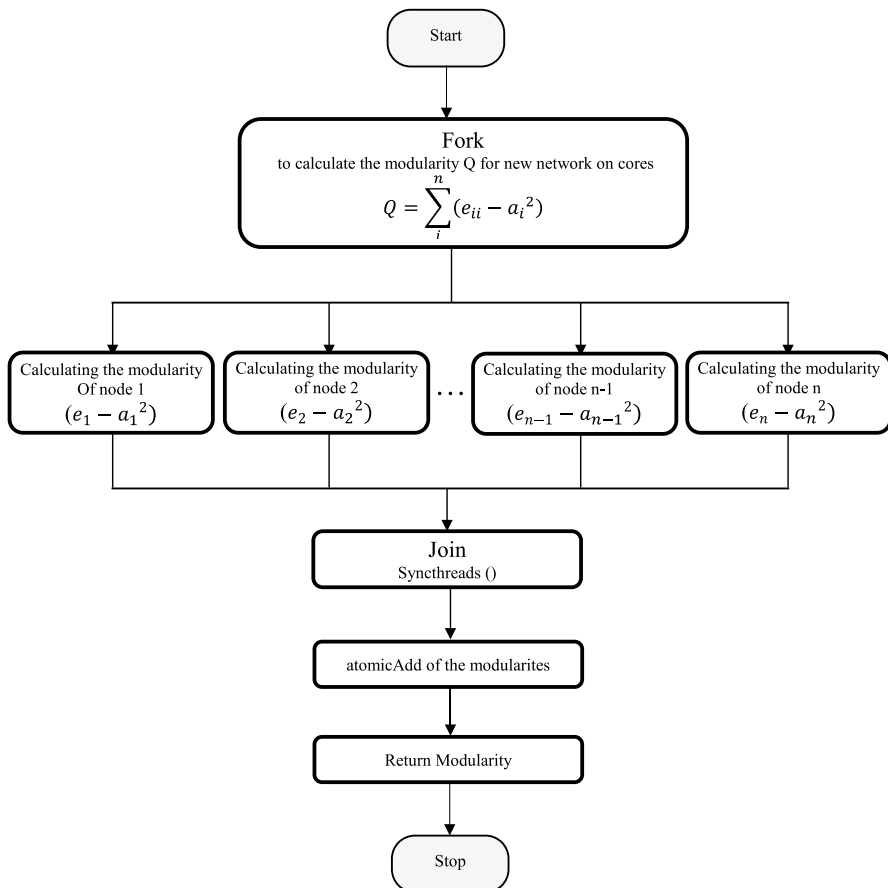


Fig. 1 The flowchart of calculating modularity using GPU cores in parallel

to find communities by Louvain method. There is a shared memory in each block that is shared among threads, and threads can use data inside the block or put the results of their calculation inside it. GPU is composed of streaming multiprocessors (SMs), each of which contains several physical processing cores. Figure 2 demonstrates an example of NVIDIA Tesla K20Xm architecture. This architecture has one GPU with 14 SMs, each of which consists of 192 hardware cores. Therefore, the overall GPU has 2688 hardware cores. Each unit of SM can have the maximum number of 2048 processing threads, and each block contains the maximum number of 1024 physical processing threads, which can execute concurrently by 192 hardware GPU cores in each SM. If the number of software threads is higher than physical processing cores, the execution of threads on SMs is concurrent. The number of registries inside SM is limited, and this hardware restriction on the data transition between threads can reduce the speed-up.

Our computational model is heterogeneous, and ACLM calculates the modularity by using SMs on GPU. Other computations execute sequentially on CPU. After

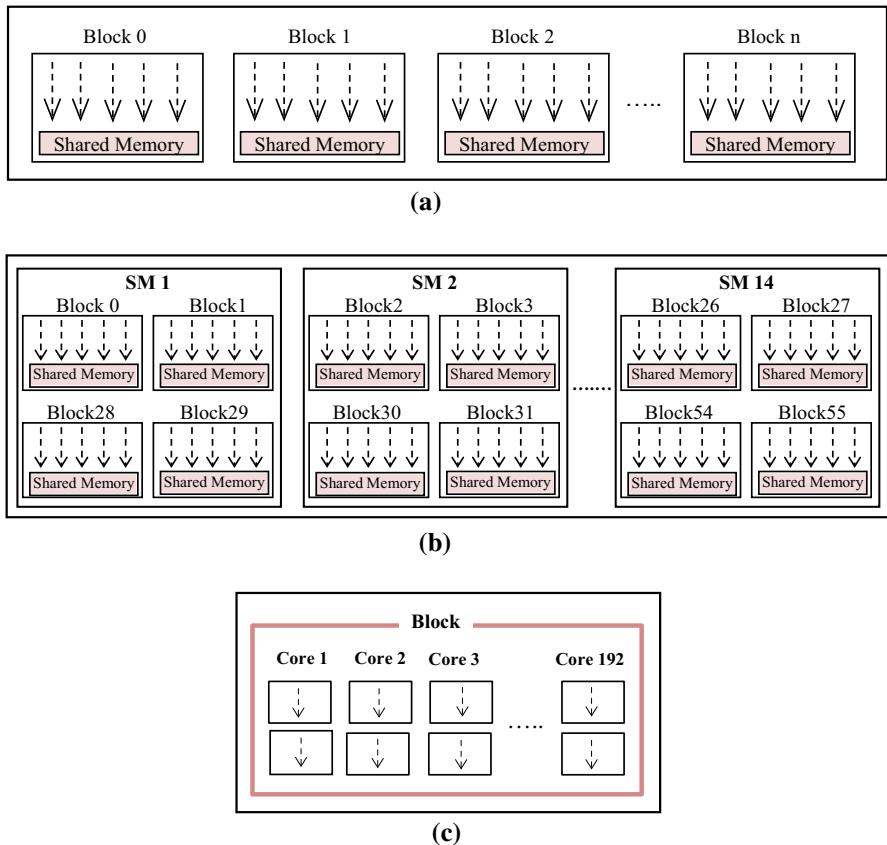


Fig. 2 Computational model for GPU blocking in NVIDIA Tesla K20Xm architecture: **a** Threads assigned to the blocks, **b** Blocks assigned to the SMs, **c** Cores assigned to the block

performing the calculations by threads in a block, they are synced using Syncthreads function. Then, the resulted modularity is stored in block-shared memory.

The calculation of other items in the modularity sigma is done by the processing threads, from the other blocks. Afterwards, the resulted modularity by threads in each block is sent to CPU and modularity is calculated by dividing the sum of the values of blocks over the sum of the weights of all edges in the graph. After running the algorithm, if modularity is increased, the supernode joins to the neighbour community. This process is performed continuously for all nodes until no further improvements can be made to the resulted modularity. Algorithms 1 to 4 describe the steps in which ACLM executes Louvain method by using the CUDA platform.

Algorithm 1: Adaptive CUDA Louvain Method (ACLM)

Input: Graph $G: (V, E)$

Result: Communities $Net_C: (V_C, E_C)$

```

01:  $Net_C \leftarrow G$ 
02:  $W_{total} \leftarrow \sum_{e \in E_C} (w_e)$ 
03:  $mod_{current} \leftarrow -\infty$ 
04: repeat
05:    $terminate \leftarrow true$ 
06:    $mod_{max} \leftarrow mod_{current}$ 
07:   for each  $v \in V$  do
08:      $v_{max} \leftarrow v$ 
09:      $size \leftarrow Degree(v)$ 
10:     for each  $(v, u) \in E$  do
11:        $V'_C \leftarrow Update(V_C, Move(community_v \xrightarrow{v} community_u))$ 
12:        $tot[] \leftarrow [v_c.tot \mid v_c \in V'_C]$  // Vector of the weighted Degree of any  $v_c \in V'_C$ 
13:        $in[] \leftarrow [v_c.in \mid v_c \in V'_C]$  // Vector of self-loops of any  $v_c \in V'_C$ 
14:        $config \leftarrow ComputeGridConfiguration(size)$ 
15:        $mod \leftarrow ComputeModularity(tot, in, W_{total}, size, config)$ 
16:       if  $(mod > mod_{max})$ 
17:          $mod_{max} \leftarrow mod$ 
18:          $v_{max} \leftarrow u$ 
19:        $terminate \leftarrow false$ 
20:        $V_C \leftarrow Update(V_C, Move(community_v \xrightarrow{v} community_{v_{max}}))$ 
21:      $mod_{current} \leftarrow mod_{max}$ 
22: until  $terminate$ 
23: Return  $Net_C$ 

```

Line 2 in Algorithm 1 stores the sum of weights for all edges in the graph in variable W_{total} . Then, for each node v in the graph, the degree of v is stored in variable $size$. At line 12, the sum of the weights for outer edges of each neighbour node is stored in tot vector. At line 13, the sum of the weights for inner edges

(self-loops) of each neighbour node is stored in the vector *in*. By calling Algorithm 2, each SM is set at line 14. Then, the modularity of the graph is calculated in Algorithm 3. In the rest of Algorithm 1, the node *v* is moved to the community for which this gain is the maximal. If not, node *v* stays in its original community. This process is applied repeatedly for all nodes until no further improvement can be achieved.

Algorithm 2: ComputeGridConfiguration

Inputs:
size // Degree of node $v \in V_G$
Result: config: (BPG, TPB, S_{SM} , N_S)

BPG // block per grid

TPB // threads per block

 S_{SM} // size of shared memory

 N_S // number of streams (needed SMs)

1: $N_{SM} \leftarrow \text{number of SMs in GPU}$ //NVIDIA Tesla K20Xm architecture has 14 SMs

2: $K \leftarrow \text{number of cores in each SM}$

3: $N_S \leftarrow \min \left\{ N_{SM}, \left\lceil \frac{\text{size}}{k} \right\rceil \right\}$

4: $s \leftarrow \frac{\text{size}}{N_S}$

5: $TPB \leftarrow \begin{cases} 32 & 1 \leq s \leq 32 \\ 64 & 32 < s \leq 64 \\ 128 & 64 < s \leq 128 \\ 256 & 128 < s \leq 256 \\ 512 & 256 < s \leq 512 \\ 1024 & \text{otherwise} \end{cases}$

6: $BPG \leftarrow \left\lceil \frac{s}{TPB} \right\rceil$

7: $S_{SM} \leftarrow TPB$

8: **Return** config

In Algorithm 2, the number of required SMs are first determined to have the optimal usage of SMs and hardware cores. To do this, the number of neighbour nodes is divided by the number of hardware cores in SM. If the number of required SMs is greater than the number of SMs in GPU, the algorithm uses the maximum number of SMs in GPU.

Because warps have fast processing times, Algorithm 2 determines the number of threads in each block based on the multiplicand of warps. If the number of required threads in SM is less than 1024, the number of threads per block is set to the two power of 5, 6, 7, 8, and 9. Otherwise, 1024 threads are allocated to each block. On lines 6 and 7, the number of blocks in each grid and the size of shared memory are set respectively.

Algorithm 3: ComputeModularity**Inputs:**

$\text{tot}[]$ // Vector of the weighted degree of any $v_j \in \{(v_i, v_j) \in E\}$
 $\text{in}[]$ // Vector of self loops of any $v_j \in \{(v_i, v_j) \in E\}$
 W_{total} // sum of the weights of all the links in the N_C
 size // Degree of v_i
 BPG // block per grid
 TPB // threads per block
 S_{SM} // size of shared memory
 N_S // number of streams (needed SMs)

Outputs:

mod // Resulting modularity

```

01:  $\text{size}_{\text{str}} \leftarrow \frac{\text{size}}{N_S}$ 
02:  $\text{BPG}_{\text{str}} \leftarrow \left\lceil \frac{\text{size}_{\text{str}}}{\text{TPB}} \right\rceil$ 
03:  $\text{streams}[] \leftarrow$  [initialize a vector of  $N_S$  streams]
04: for each  $i \in [0, N_S - 1]$  do
05:    $\text{tot}_{\text{str}}[i] \leftarrow \text{tot}[i \times \text{size}_{\text{str}}, (i + 1) \times \text{size}_{\text{str}}]$ 
06:    $\text{in}_{\text{str}}[i] \leftarrow \text{in}[i \times \text{size}_{\text{str}}, (i + 1) \times \text{size}_{\text{str}}]$ 
07:  $\text{config} \leftarrow (\text{BPG}_{\text{str}}, \text{TPB}, S_{SM}, \text{streams})$  // Kernel configuration
08:  $\text{quality} \leftarrow \text{ModularityKernel}(\langle\langle\text{config}\rangle\rangle)(\text{tot}_{\text{str}}, \text{in}_{\text{str}}, W_{\text{total}}, \text{size}_{\text{str}})$ 
09:  $\text{cudaStreamSynchronize}(\text{streams})$ 
10:  $\text{mod} \leftarrow \frac{\sum_{i \in [0, N_S - 1]} (\text{quality}[i])}{W_{\text{total}}}$ 
11: Return  $\text{mod}$ 
  
```

In our method, for the calculation of modularity, the items in modularity sigma are distributed among SMs. On lines 4 to 6 in Algorithm 3, items of the modularity sigma are allocated to each SM and CUDA kernel is configured at line 7. Then the kernel is called with parameters at line 8. After finishing the computation of modularity in SMs, SMs are synced at line 9. The modularity value is achieved at line 10 by dividing the sum of calculated modularity values in SMs over the sum of weights for all edges in the graph.

Algorithm 4: ModularityKernel**Inputs:**

$\text{tot_str}[]$ // Vector of the weighted degree of any $v_i \in \{(v_i, v_j) \in E\}$ assigned to current SM
 $\text{in_str}[]$ // Vector of self loops of any $v_i \in \{(v_i, v_j) \in E\}$ assigned to current SM
 W_{total} // sum of the weights of all the links in the N_c
 size_str // The number of v_i 's neighbours assigned to current SM
 stream // current stream

Output:

$\text{quality}[]$ // Modularity of current SM

```

01:  $\text{cache}[] \leftarrow$  define an array of type extern _shared_ float // Shared memory
02:  $\text{tid} \leftarrow \text{threadIdx} + \text{blockIdx} \times \text{blockDim}$ 
03:  $\text{tmp} \leftarrow 0$ 
04: while ( $\text{tid} < \text{size}$ ) do
05:    $\text{temp} \leftarrow \text{temp} + \text{in}[\text{tid}] - \frac{(\text{tot\_str}[\text{tid}])^2}{W_{\text{total}}}$ 
06:    $\text{tid} \leftarrow \text{tid} + \text{blockDim} \times \text{gridDim}$ 
07:  $\text{cache}[\text{threadIdx}] \leftarrow \text{temp}$ 
08:  $\text{syncthreads}()$  // To guarantee that every thread in the block is completed
09:  $\text{quality}[\text{stream}] \leftarrow \sum_{\text{atomic}}(\text{cache})$  // (atomicAdd in CUDA) Sum of values in the cache
10: return quality
  
```

In Algorithm 4, on lines 4 to 6, items of modularity sigma are calculated. After finishing the computation of modularity by threads in the block, on line 8, threads are synced. At line 9, the sum of the value for calculated modularity in blocks is stored in the *quality* vector, and the modularity value in an SM is achieved.

Prediction of the number of needed threads in each block to calculate the modularity sigma is the key to reduce parallelization overhead and accelerate the algorithm execution time. In the next section, we will see the results of this idea.

5 Experimental results

We compared our proposed method with three parallel methods for Louvain community detection algorithms. The first one is PLM [28], which calculates the sigma of the modularity of adding a neighbour node in parallel. The second method is APLM [11] that calculates the suitable number of processing threads for the computation of modularity for each two neighbour nodes adaptively. The third one is NAIM a GPU-based algorithm uses hardware cores to calculate modularity [31]. Tables 1 and 2 demonstrate the specifications of our implementation platforms, which is used for all the experiments.

Table 1 Platform for experiments

CPU	AMD Opteron(tm) Processor 6344
GPU	NVIDIA Tesla K20Xm
RAM	128 GB
OS	Linux: CentOS release 6.3 (Final)
Compiler	– nvcc: NVIDIA (R) Cuda compiler driver – gcc 4.8.2
Cuda compilation tools	Release 6.5, V6.5.12

Table 2 Platform 2 for experiments

CPU	Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40 GHz
GPU	Nvidia Geforce GTX 1080 ti
RAM	64 GB
OS	Linux: Ubuntu server 18.04 LTS
Compiler	– nvcc: NVIDIA (R) Cuda compiler driver – gcc version 7.4.0
Cuda compilation tools	Release 10.0, V10.0.130

We have used CUDA platforms to execute the proposed ACLM algorithm and the NAIM on GPU. NVCC compiler was used for compiling the source codes. Nvidia's latest CUDA provides a variety of synchronization methods. The characteristics of the CUDA version (10.0) are explained in [33].

In order to compare our results with APLM and PLM algorithms, these projects were run in CPU on GCC compiler with version 4.8.2 on Platform 1 and GCC compiler with version 7.4.0 on Platform 2. The community detection algorithms were implemented with C++. Complex networks of the real-world contain heterogeneous data, and this means choosing an ideal or general sample data and then generalizing it to be impossible. In this research, four benchmarks have been used whose details are presented in Table 3. The graphs have different dimensions from small size like CNR-2000 to large-scale graphs like Cage-15. The real networks (the graphs were extracted from real virtual networks) of the world contain heterogeneous data, and this leads to the fact that choosing an ideal sample data or a general sample data and then generalizing it is impossible. The graphs have a different ratio of edges divided over the nodes. Therefore, the execution time of the algorithm differs for the graphs.

Calculation of modularity on different benchmarks includes a number of passes (levels). We have extracted the number of nodes in each passing of ACLM algorithm on the Benchmarks. The results are shown in Table 4. Because in Louvain at first, each node in the network is considered a community. The maximum number of supernodes that equals the size of the networks occurs at the first pass of the algorithm. In the next passes, the size of the network decreases.

The NVIDIA Tesla K20Xm architecture has one GPU with 14 SMs, each of which consists of 192 hardware cores. Each block contains the maximum number

Table 3 Specifications of the benchmark graphs

Benchmark	# of nodes	# of links	Average degree	Maximum indegree	Maximum outdegree	Description
CNR-2000	325,557	2,738,969	9.879	18,235	2716	A small crawl of the Italian CNR domain
EU-2005	862,664	16,138,468	22.297	68,922	6985	A small crawl of the.eu domain
IN-2004	1,382,908	13,591,473	12.233	21,866	7753	A small crawl of the.in domain performed for the Nagaoka University of Technology
Cage-15	5,154,859	47,022,346	18.24	46	–	DNA electrophoresis, 15 monomers in polymer. A. van Heukelum, Utrecht U

Table 4 Size of the graph (# node) in the different levels on the Benchmarks in the ACLM algorithm

Level number/ benchmark	Level 0	Level 1	Level 2	Level 3	Level 4	Level 5	Level 6	Level 7
CNR-2000	325,557	19,516	3094	929	370	250	235	–
EU-2005	862,664	23,976	1226	380	329	322	–	–
IN-2004	1,382,906	88,991	14,506	3258	1387	1038	1012	1009
Cage-15	5,154,860	4,929,449	4,893,069	4,890,847	4,890,587	4,890,524	4,890,499	4,890,497

of 1024 processing threads, which can execute concurrently by the 192 hardware cores in each SM. In the ACLM algorithm, at first, the number of required SMs are calculated. For the optimal usage of SMs and hardware cores, the number of neighbour nodes is divided by 192 cores in SM, and the number of required SMs is determined. If the number of required SMs is greater than 14, the algorithm uses the maximum number of SMs in GPU (14). Then to determine the number of required threads, the number of neighbour nodes is divided by the number of required SMs.

If the number of threads in SM is less than 32, 32 threads are allocated to the block. If it is between 32 and 64 threads, 64 threads are allocated to the block. If it is between 65 and 128 threads, 128 threads are allocated; if it is between 129 and 256 threads, 256 threads are allocated; if it is between 257 and 512 threads, 512 threads are allocated to each block. Otherwise, 1024 threads are allocated to each block. The number of blocks on GPU is determined by dividing the number of neighbouring nodes by the number of threads per block. Table 5 demonstrates the number of SMs, blocks and threads per block in ACLM algorithm for different levels of the algorithm in benchmarks. As you can see in this table, by increasing the size of the graph, the number of allocated threads in each block is increased. Hence, it is important to adaptively calculate it in the algorithm.

NAIM uses a full block or a fraction of one physical warp, like its execution unit, when it assigns tasks to GPU threads. It divides the vertices in the graph to seven different groups. The first six groups contain all vertices with degrees in the range [1, 4], [5, 8], [9, 16], [17, 32], [33, 84], and [85, 319], respectively, while the last group contains all vertices with the degree higher than 319. For the groups which $k=1$ through 4, it assigns 2^{k+1} threads to each vertex. All threads assigned to the same vertex belong to the same warp. A vertex in the group five is allocated to all threads of one warp. In groups six and seven, all the threads (128) in a block are assigned to each vertex. In this way, NAIM tries to use the blocks in the GPU.

5.1 Comparison of computation times

In this section, we evaluate the methods based on algorithm execution time. Execution times of the proposed ACLM in addition to PLM, APLM and NAIM are shown in Tables 6 and 7 for Platform 1 and Platform 2, respectively.

Table 5 Number of SMs, blocks and threads per block in the different levels of ACLM algorithm

Level number/benchmark		CNR-2000	EU-2005	IN-2004	Cage-15
Level 0	# SM	14	14	14	14
	# blocks	318	843	1351	5035
	# threads per block	1024	1024	1024	1024
Level 1	# SM	14	14	14	14
	# blocks	20	24	87	4814
	# threads per block	1024	1024	1024	1024
Level 2	# SM	14	7	14	14
	# blocks	13	5	15	4779
	# threads per block	256	256	1024	1024
Level 3	# SM	5	2	14	14
	# blocks	4	2	13	4777
	# threads per block	256	256	256	1024
Level 4	# SM	2	2	8	14
	# blocks	2	2	6	4776
	# threads per block	256	256	256	1024
Level 5	# SM	2	2	6	14
	# blocks	2	2	5	4776
	# threads per block	128	256	256	1024
Level 6	# SM	2	–	6	14
	# blocks	2	–	4	4776
	# threads per block	128	–	256	1024
Level 7	# SM	–	–	6	14
	# blocks	–	–	4	4776
	# threads per block	–	–	256	1024

Table 6 Execution time (Second) of applying algorithms on the benchmarks in Platform 1

Benchmarks/algorithms	PLM	APLM	NAIM	ACLM
CNR-2000	2.74	0.15	0.4	0.10
EU-2005	3.69	0.49	1.32	0.31
IN-2004	3.71	0.60	0.95	0.47
Cage-15	36.88	10.17	13.1	8.32

Table 7 Execution time (Second) of applying algorithms on the benchmarks in Platform 2

Benchmarks/algorithms	PLM	APLM	NAIM	ACLM
CNR-2000	0.78	0.12	0.11	0.09
EU-2005	2.1	0.36	0.34	0.28
IN-2004	1.65	0.49	0.42	0.36
Cage-15	14.52	9.14	–	8.01

Considering the results shown in Table 6, in Platform 1, the execution time of the proposed ACLM algorithm by using 2688 CUDA processing cores is better than other algorithms. Reduction in the execution time of this algorithm in comparison with PLM is 96% in benchmark CNR-2000, 91% in benchmark EU-2005 and 87% in benchmark IN-2004 and 77% in benchmark Cage-15. Also, the reduction in execution time of ACLM in comparison with APLM is 33% in CNR-2000 benchmark, 37% in EU-2005 benchmark, 22% in IN-2004 benchmark and 18% in Cage-15 benchmark. These results indicate that the parallelization overhead of using GPU, especially communication overhead, in the proposed ACLM is acceptable and it overcomes the implementations of multicore platforms.

In platform 1, reduction in the execution time of ACLM algorithm in comparison to NAIM is improved 75% in CNR-2000 benchmark, 77% in EU-2005 benchmark, 51% in IN-2004 benchmark and 36% in Cage-15 benchmark. This indicates the proposed GPU-based implementation is better than NAIM. The reason behind this is dynamically allocation of the cores inside GPU in the proposed ACLM algorithm.

In Platform 2, considering the results shown in Table 7, the execution times of the GPU-based implementations are better than a multicore implementation for all benchmarks. This indicates that using GPU to accelerate Louvain community detection is an appropriate technique with modern GPU platforms with huge numbers of cores. As you can see here, the proposed ACLM algorithm again outperforms previous GPU-based implementation NAIM for all benchmarks. This indicates the effectiveness of adaptively allocating cores in SMs to the hardware threads besides using less memory usage than NAIM. Also, NAIM cannot overcome APLM (which is a multicore implementation) at least for three benchmarks. The main reason behind this is the communication overhead of the GPU platform.

In Platform 2, the execution times of the ACLM algorithm in comparison with NAIM is improved. This indicates that the proposed GPU-based implementation is better than the previous one in a newer GPU. The reason behind this is the adaptive allocation of the cores inside GPU and supporting this idea in new GPUs. We could not run NAIM for the biggest benchmark graph on this platform, because of its memory usage. However, the proposed ACLM can run and finish on this platform. Experiments have shown that shared memory has not been used efficiently in NAIM. Also, for the large benchmarks, there is a lack of memory usage, and NAIM stops to work.

The results indicate that using GPU to accelerate Louvain community detection is an appropriate technique in modern GPU platforms with a huge number of cores. For big graphs, employing more threads leads to using more cores in SMs and reducing execution time, although synchronization overhead is increased.

5.2 Comparison of modularity amounts

While the proposed ACLM algorithm leads to accelerate Louvain method, the results in Tables 8 and 9 indicate that the modularity resulted from executing this algorithm on the CUDA platform is comparable to the results from other algorithms.

Table 8 Modularity of applying algorithms on the benchmarks in Platform 1

Benchmarks/algorithms	PLM	APLM	NAIM	ACLM
CNR-2000	0.912728	0.912667	0.912909	0.912807
EU-2005	0.938786	0.937441	0.936152	0.937365
IN-2004	0.98031	0.980058	0.978909	0.979802
Cage-15	0.893829	0.866638	0.850144	0.866638

Table 9 Modularity of applying algorithms on the benchmarks in Platform 2

Benchmarks/algorithms	PLM	APLM	NAIM	ACLM
CNR-2000	0.912812	0.912667	0.912909	0.912807
EU-2005	0.938533	0.937441	0.936152	0.937365
IN-2004	0.980364	0.980058	0.978909	0.979802
Cage-15	0.893814	0.866638	–	0.866638

As is depicted in Tables 8 and 9, all algorithms have near similar modularity results, as they have the same nature. Since all algorithms basis is hierarchical clustering, they have the same behaviour. The results show that the PLM algorithm has relatively better modularity on benchmarks, and the modularity of the ACLM algorithm is near to the APLM algorithm. Generally, these algorithms are resistant to any change on the modularity, and the resulted communities have similar modularity amount. Compared to NAIM, the best modularity result for the ACLM algorithm is obtained in Cage-15 benchmark, which is the largest graph in benchmarks. Therefore, the results indicate the proposed adaptive thread assignment can also reach comparable modularity compared to previous algorithms for large-scale graphs.

6 Conclusions

In this research, we proposed a GPU based community detection algorithm to accelerate the Louvain method called Adaptive CUDA Louvain method (ACLM). The algorithm uses shared memory space inside blocks of GPU and the optimal number of threads to calculate modularity sigma in parallel. ACLM calculates the number of SMs required to calculate the modularity on GPU, before assigning hardware threads. consequently, it determines the number of threads in each block based on the multiplicand of warps. Because the threads in a warp on GPU have faster processing times, compared to the threads in two or more number of warps. The results of running ACLM on several web-based networks demonstrate that it has an improvement up to 77% in comparison with the previous parallel Louvain algorithms. The results show that solely increasing the number of threads in blocks does not lead to an increase in algorithm execution time, and it depends on the number of hardware cores on GPU as well. The scalability of the ACLM algorithm is limited

by the capability of GPU. In future work, we plan to improve our algorithm by using multi-GPU platforms.

References

1. Guendouz M, Amine A, Hamou RM (2017) discrete modified fireworks algorithm for community detection in complex networks. *Appl Intell* 46:373–385. <https://doi.org/10.1007/s10489-016-0840-9>
2. Fortunato S (2010) Community detection in graphs. *Phys Rep* 486:75–174. <https://doi.org/10.1016/j.physrep.2009.11.002>
3. Newman MEJ, Girvan M (2004) Finding and evaluating community structure in networks. *Phys Rev E* 69:026113. <https://doi.org/10.1103/PhysRevE.69.026113>
4. Clauset A, Newman MEJ, Moore C (2004) Finding community structure in very large networks. *Phys Rev E* 70:066111. <https://doi.org/10.1103/PhysRevE.70.066111>
5. Brandes U, Delling D, Gaertler M, Gorke R, Hoefer M, Nikoloski Z, Wagner D (2008) On modularity clustering. *IEEE Trans Knowl Data Eng* 20:172–188. <https://doi.org/10.1109/TKDE.2007.190689>
6. Blondel VD, Guillaume JL, Lambiotte R, Lefebvre E (2008) Fast unfolding of communities in large networks. *J Stat Mech* 10:P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
7. Moradi E, Fazlali M, Tabatabaee Malazi H (2015) Fast parallel community detection algorithm based on modularity. In: 2015 18th CSI international symposium on computer architecture and digital systems (CADSD). IEEE. <https://doi.org/10.1109/CADS.2015.7377794>
8. Staudt CL, Meyerhenke H (2013) Engineering parallel algorithms for community detection in massive networks. In: Proceedings of the 42nd International Conference on Parallel Processing. <https://doi.org/10.1109/ICPP.2013.27>
9. Cheong CY, Huynh HP, Lo D, Goh RSM (2013) Hierarchical parallel algorithm for modularity-based community detection using GPUs. In: Proceedings of the 19th International Conference on Parallel Processing, Euro-Par'13. Springer, Berlin, pp 775–787. https://doi.org/10.1007/978-3-642-40047-6_77
10. Lu H, Halappanavar M, Kalyanaraman A (2015) Parallel heuristics for scalable community detection. *Parallel Comput* 47:19–37. <https://doi.org/10.1016/j.parco.2015.03.003>
11. Fazlali M, Moradi E, Tabatabaee Malazi H (2017) Adaptive parallel Louvain community detection on a multicore platform. *Microprocess Microsyst* 54:26–34. <https://doi.org/10.1016/j.micpro.2017.08.002>
12. Zeng J, Yu H (2018) A scalable distributed Louvain algorithm for large-scale graph community detection. In: 2018 IEEE International Conference on Cluster Computing (CLUSTER). <https://doi.org/10.1109/CLUSTER.2018.00044>
13. Forster R (2016) Louvain community detection with parallel heuristics on GPUs. In: IEEE 20th Jubilee International Conference on Intelligent Engineering Systems (INES), Budapest, Hungary, 30 June–2 July. <https://doi.org/10.1109/INES.2016.7555126>
14. Erdos P, Renyi A (1960) On the evolution of random graphs. *Publ Math Inst Hung Acad Sci* 5:17–61
15. Rezapoor Mirsaleh M, Meybodi MR (2016) A Michigan memetic algorithm for solving the community detection problem in complex network. *Neurocomputing* 214:535–545. <https://doi.org/10.1016/j.neucom.2016.06.030>
16. Newman MEJ (2006) Modularity and community structure in networks. *Proc Natl Acad Sci* 103:8577–8582. <https://doi.org/10.1073/pnas.0601602103>
17. Holmström E, Bock N, Brännlund J (2009) Modularity density of network community divisions. *Physica D* 238:1161–1167. <https://doi.org/10.1016/j.physd.2009.03.015>
18. Zhao Y (2017) A survey on theoretical advances of community detection in networks. *WIREs Comput Stat* e1403:1–13. <https://doi.org/10.1002/wics.1403>
19. Khan BS, Niazi MA (2017) Network community detection: a review and visual survey. arXiv preprint: arXiv:1708.00977
20. Chakraborty T, Dalmia A, Mukherjee A, Ganguly N (2017) Metrics for community analysis: a survey. *ACM Comput Surv* 50:1–37. <https://doi.org/10.1145/3091106>

21. Ghosh S, Halappanavar M, Tumeo A, Kalyanaraman A, Lu H, Chavarria-Miranda D, Khan A, Gebremedhin A (2018) Distributed louvain algorithm for graph community detection. In: 2018 IEEE international parallel and distributed processing symposium (IPDPS), pp 885–895. <https://doi.org/10.1109/IPDPS.2018.00098>
22. Ozaki N, Tezuka H, Inaba M (2016) A simple acceleration method for the Louvain algorithm. *Int J Comput Electr Eng* 8:207–218. <https://doi.org/10.17706/ijcee.2016.8.3.207-218>
23. Lancichinetti A, Fortunato S, Radicchi F (2008) Benchmark graphs for testing community detection algorithms. *Phys Rev E* 78:046110. <https://doi.org/10.1103/PhysRevE.78.046110>
24. Gutiérrez I, Gómez D, Castro J, Espínola R (2019) A new community detection algorithm based on fuzzy measures. In: International Conference on Intelligent and Fuzzy Systems, pp 133–140. https://doi.org/10.1007/978-3-030-23756-1_18
25. Traag VA (2015) Faster unfolding of communities: speeding up the Louvain algorithm. *Phys Rev E* 92:032801. <https://doi.org/10.1103/PhysRevE.92.032801>
26. Carnivali GS, Vieira AB, Ziviani A, Esquef PAA (2020) CoVeC: coarse-grained vertex clustering for efficient community detection in sparse complex networks. *Inf Sci*. <https://doi.org/10.1016/j.ins.2020.03.004>
27. Bhowmick S, Srinivasan S (2013) A template for parallelizing the louvain method for modularity maximization. *Dyn Complex Netw* 2:111–124. https://doi.org/10.1007/978-1-4614-6729-8_6
28. Staudt CL, Meyerhenke H (2016) Engineering parallel algorithms for community detection in massive networks. *Parallel Distrib Syst IEEE Trans* 27:171–184. <https://doi.org/10.1109/TPDS.2015.2390633>
29. Raghavan UN, Albert R, Kumara S (2007) Near linear time algorithm to detect community structures in large-scale networks. *Phys Rev E* 76:036106. <https://doi.org/10.1103/PhysRevE.76.036106>
30. Shao J, Han Z, Yang Q, Zhou T (2015) Community detection based on distance dynamics. In: Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, New York, pp 1075–1084. <http://dx.doi.org/https://doi.org/10.1145/2783258.2783301>
31. Naim Md, Manne F, Halappanavar M, Tumeo A (2017) Community detection on the GPU. In: 2017 IEEE international parallel and distributed processing symposium. <https://doi.org/10.1109/IPDPS.2017.16>
32. Newman MEJ (2004) Fast algorithm for detecting community structure in networks. *Phys Rev E* 69:066133. <https://doi.org/10.1103/PhysRevE.69.066133>
33. Zhang L, Wahib M, Zhang H, Matsuoka S (2020) A study of single and multi-device synchronization methods in Nvidia GPUs. In: IEEE international parallel & distributed processing symposium 2020.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.